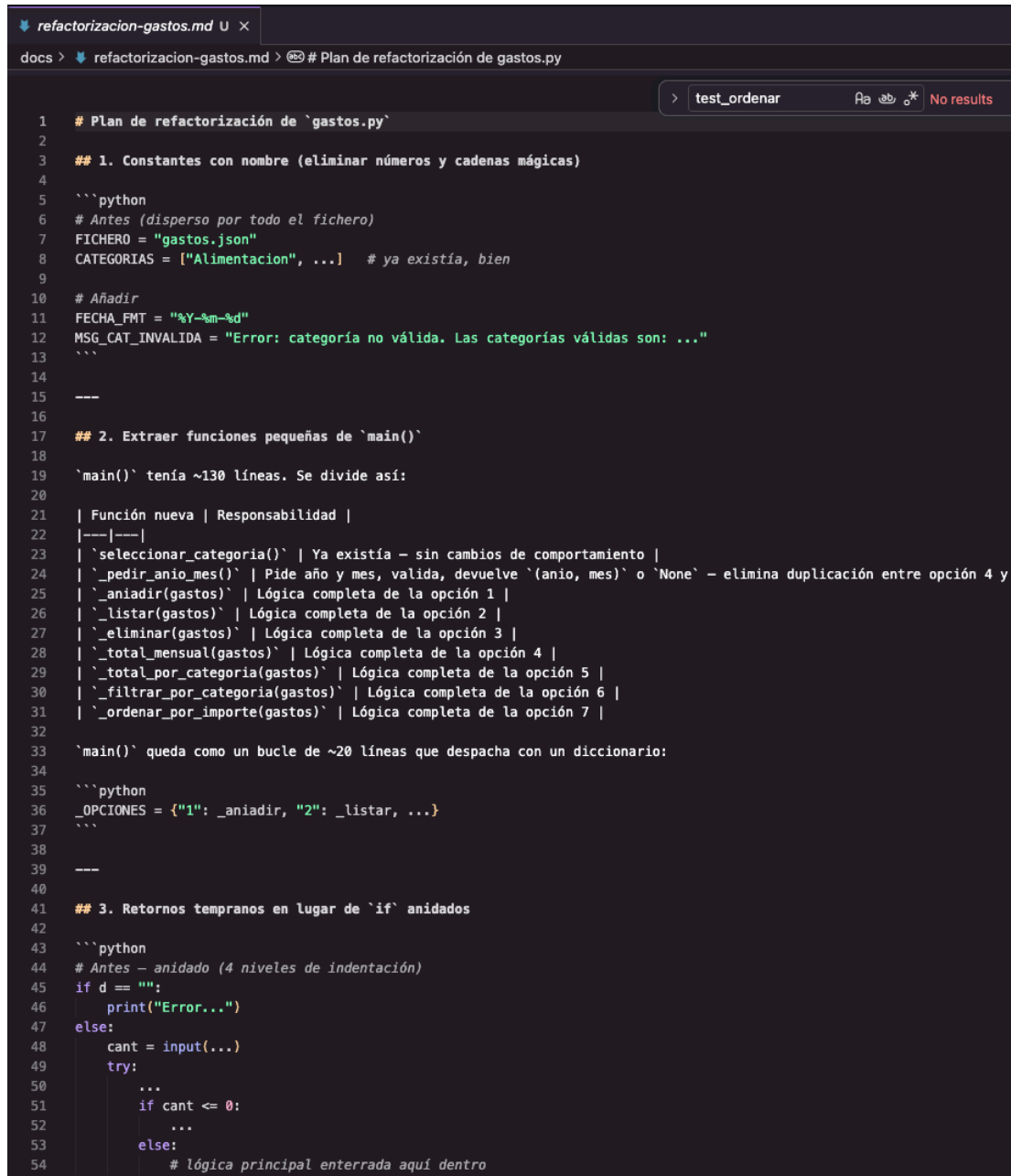


Ejercicio práctico. Refactorización de un módulo legacy con asistencia de IA.

De nuevo, continuamos con el mismo proyecto sencillo desarrollado para estas prácticas entregables.

Se decide refactorizar el módulo principal del proyecto gastos.py, desde su estado inicial.

Inicialmente se pide un plan de refactorización detallado a Kiro y que genere un documento detallado que podamos añadir al proyecto:



```
refactorizacion-gastos.md x
docs > refactorizacion-gastos.md # Plan de refactorización de gastos.py

> test_ordenar  No results

1  # Plan de refactorización de `gastos.py`
2
3  ## 1. Constantes con nombre (eliminar números y cadenas mágicas)
4
5  ```python
6  # Antes (disperso por todo el fichero)
7  FICHERO = "gastos.json"
8  CATEGORIAS = ["Alimentacion", ...] # ya existía, bien
9
10 # Añadir
11 FECHA_FMT = "%Y-%m-%d"
12 MSG_CAT_INVALIDA = "Error: categoría no válida. Las categorías válidas son: ..."
13 ```
14
15 ---
16
17 ## 2. Extraer funciones pequeñas de `main()`
18
19 `main()` tenía ~130 líneas. Se divide así:
20
21 | Función nueva | Responsabilidad |
22 |---|---|
23 | `seleccionar_categoria()` | Ya existía - sin cambios de comportamiento |
24 | `_pedir_anio_mes()` | Pide año y mes, valida, devuelve `(anio, mes)` o `None` - elimina duplicación entre opción 4 y
25 | `_aniadir(gastos)` | Lógica completa de la opción 1 |
26 | `_listar(gastos)` | Lógica completa de la opción 2 |
27 | `_eliminar(gastos)` | Lógica completa de la opción 3 |
28 | `_total_mensual(gastos)` | Lógica completa de la opción 4 |
29 | `_total_por_categoria(gastos)` | Lógica completa de la opción 5 |
30 | `_filtrar_por_categoria(gastos)` | Lógica completa de la opción 6 |
31 | `_ordenar_por_importe(gastos)` | Lógica completa de la opción 7 |
32
33 `main()` queda como un bucle de ~20 líneas que despacha con un diccionario:
34
35 ```python
36 _OPCIONES = {"1": _aniadir, "2": _listar, ...}
37 ```
38
39 ---
40
41 ## 3. Retornos tempranos en lugar de `if` anidados
42
43 ```python
44 # Antes - anidado (4 niveles de indentación)
45 if d == "":
46     print("Error...")
47 else:
48     cant = input(...)
49     try:
50         ...
51         if cant <= 0:
52             ...
53         else:
54             # lógica principal enterrada aquí dentro
55     except:
56         ...
```

Realiza toda la reestructuración de manera automatizada y rápida, exponiendo un detalle al finalizar:

```
Esto es lo que cambió en el código:
```

Estructura

- `main()` pasó de ~130 líneas con 7 bloques `if/elif` a un bucle de 20 líneas que despacha a través del diccionario `_OPCIONES`.
- Se extrajeron 9 funciones: una por cada opción del menú más `_pedir_anio_mes()` que elimina la duplicación entre las opciones 4 y 5.

Legibilidad

- `l2, d, cant, cat, tmp` → `gastos, descripcion, cantidad, categoria, opcion`.
- `if anidados` → retornos tempranos (`return` al primer error, sin `else`).
- Validación de fecha: `split("-")` manual → `datetime.strptime(fecha_str, FECHA_FMT)`.

Constantes

- `FICHERO, FECHA_FMT, MSG_CAT_INVALIDA` sustituyen las cadenas repetidas.
- Type hints y docstrings de una línea en todas las funciones.

Est. Credits Used: 0,97 Elapsed time: 1m

Todo está detallado en el repositorio: